SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4

Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

SIMATS ENGINEERING

(Saveetha School of Engineering)
Department of Computer Science and Engineering

CI/CD PIPELINE DESIGN DOCUMENT

Intelligent AI-Based Smart Public Transportation Demand Forecasting System

Student Name	RAMYA .N	**Register No.**	192511403
Course Code	CSA1014	**Course Title**	Software Engineering
CO Assessed	CO4	**Assessment**	Assessment Tool 2 – CI/CD Pipeline Design
Assigned Problem	20. Intelligent AI-Based Smart Public Transportation Demand Forecasting System		

1. Introduction & Industry Context

Public transportation agencies are under constant pressure to deliver reliable, efficient, and commuter-friendly services while coping with continuously fluctuating passenger demand. Conventional scheduling methods rely on static, historically averaged timetables that cannot react to real-time variables such as weather disruptions, special events, or sudden traffic congestion. This leads to a recurring cycle of overcrowded buses/trains during peak hours and wasted capacity during off-peak periods, ultimately harming service reliability, commuter satisfaction, and operational cost-efficiency.

The proposed Intelligent AI-Based Smart Public Transportation Demand Forecasting System addresses this gap by combining Artificial Intelligence, IoT sensor data, GPS telemetry, and Big Data Analytics into a single predictive platform. The system continuously ingests historical ridership records, live weather feeds, city-event calendars, and traffic-pattern data to forecast passenger demand at a route and time-slot level. These forecasts drive automated schedule optimization, dynamic fleet allocation, and real-time passenger information updates, directly enabling the objectives of improved reliability, reduced waiting time, better fleet utilization and sustainable urban mobility.

Because the system directly influences live transit operations and is updated frequently as new ridership data and improved forecasting models become available, a robust, automated CI/CD (Continuous Integration / Continuous Deployment) pipeline is essential. It allows the engineering and data-science teams to integrate code and model changes safely, validate them thoroughly, and deploy them to production with minimal risk and near-zero downtime.

2. Requirement & CI/CD Pipeline Analysis

2.1 Functional Requirements Driving CI/CD Needs

- Frequent retraining and redeployment of the AI/ML demand-forecasting model as new ridership and weather data arrive.
- Multiple microservices — data ingestion, model inference API, schedule optimizer, and operator dashboard — must be built, tested and released independently.
- Real-time IoT/GPS data streams require the pipeline to validate schema and latency-sensitive integrations before every release.
- High availability is mandatory: forecasting and scheduling APIs cannot go down during peak commuting hours, so deployments must be zero/near-zero downtime.
- Frequent small releases (agile data-science iteration) demand fast feedback loops — automated build and test execution within minutes of a commit.

2.2 Non-Functional / Operational CI/CD Requirements

- Scalability: pipeline and infrastructure must handle autoscaling for sudden demand spikes (e.g., festival/event traffic).

- Security & Privacy: commuter GPS and ridership data must be protected through the pipeline (secrets management, dependency scanning, encryption).
- Auditability: every release must be traceable to a commit, tester, and approver for regulatory and operational accountability.
- Reliability: automated rollback and health monitoring must prevent a faulty model or service from disrupting live operations.
- Speed: developers and data scientists need fast feedback — target build+test cycle under 10 minutes.

2.3 CI/CD Goals for This Project

1. Automate integration of code from multiple contributors (backend, ML, frontend dashboard) into a single verified build.
2. Continuously validate the forecasting model's accuracy and the platform's functional correctness before every release.
3. Automatically build, containerize and push versioned deployable artifacts.
4. Progressively deploy through Development → Staging → Production with quality gates at each step.
5. Provide real-time monitoring, alerting, and automatic rollback to guarantee continuous public-facing service availability.

2.4 Assumptions & Constraints

- The transit agency already owns GPS-enabled fleet vehicles and IoT occupancy sensors that stream data over MQTT.
- Historical ridership data (minimum 2 years) is available in a structured data warehouse for model training and validation.
- The organization operates on a cloud provider (AWS assumed here) supporting managed Kubernetes (EKS), ECR, and CloudWatch/Prometheus integration.
- Regulatory constraints require passenger location data to be anonymized in any non-production environment.
- The team follows an agile workflow with releases planned on a bi-weekly sprint cadence, supplemented by hotfix releases as needed.

3. System Architecture Overview

Before designing the CI/CD pipeline itself, it is important to understand the overall system that the pipeline builds, tests, and deploys. The platform follows a layered, microservices-based architecture that separates data acquisition, AI processing, and consumer-facing delivery, allowing each layer to be independently versioned, tested, and released through the pipeline.

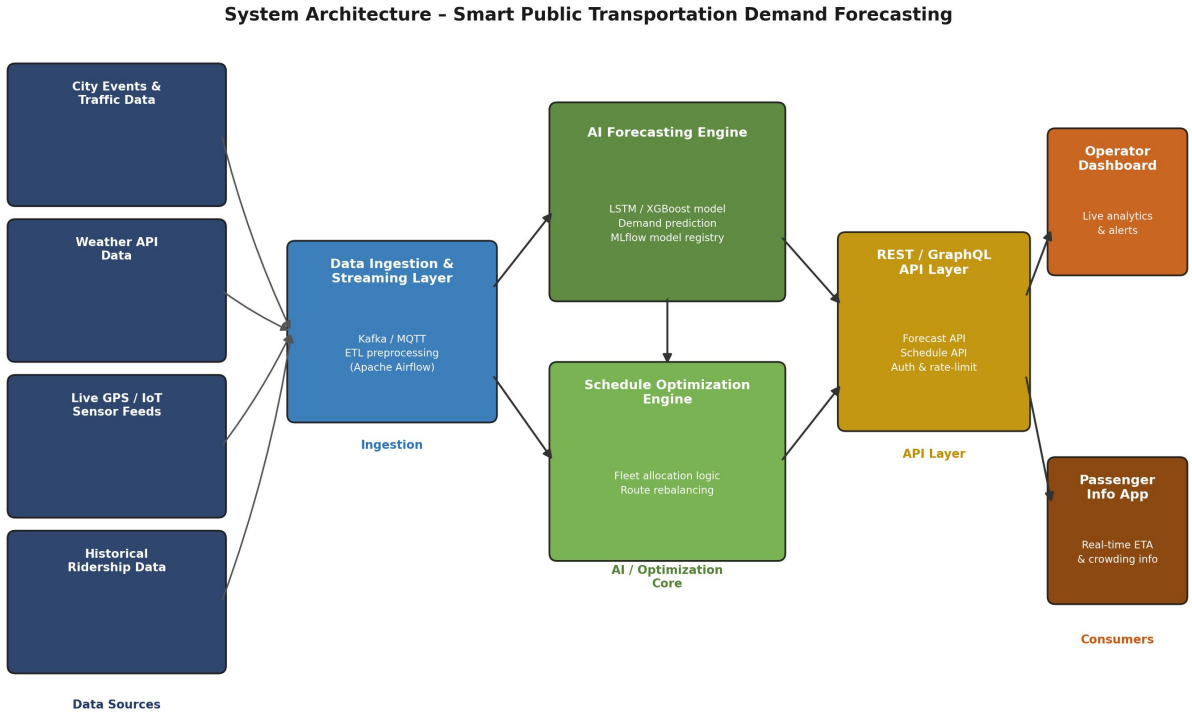

Figure 1: High-Level System Architecture – Smart Public Transportation Demand Forecasting System

3.1 Architecture Layers

Data Source Layer

Feeds the platform with historical ridership records, live GPS/IoT sensor streams from buses and trains, third-party weather API data, and city event/traffic feeds. Each source has its own ingestion adapter, so the pipeline can test and version these adapters independently.

Ingestion & Streaming Layer

A Kafka/MQTT-based streaming layer, orchestrated by Apache Airflow, collects, cleans, and buffers incoming data before it reaches the AI core. This decoupling ensures that a spike in sensor traffic does not directly overload the forecasting service.

AI / Optimization Core

The AI Forecasting Engine (LSTM/XGBoost ensemble, tracked via an MLflow model registry) predicts route-level and time-slot-level passenger demand. Its output feeds the Schedule Optimization Engine, which converts demand forecasts into concrete fleet allocation and route-rebalancing recommendations.

API Layer

A REST/GraphQL API layer exposes forecast and schedule data securely to downstream consumers, handling authentication and rate-limiting so the AI core is insulated from direct external traffic.

Consumer Layer

The Operator Dashboard gives transit planners live analytics and alerts, while the Passenger Info App surfaces real-time ETA and crowding information to commuters — both are independently deployable front-end services within the same pipeline.

Because each layer is containerized as an independent microservice, the CI/CD pipeline described in Section 4 can build, test and deploy them individually or together, giving the team fine-grained control over release scope and risk.

4. Pipeline Architecture & Workflow

The CI/CD pipeline for the Smart Public Transportation Demand Forecasting System is structured as a ten-stage automated workflow, beginning the moment a developer commits code and ending with live, monitored production traffic. Each stage acts as a quality gate — code only progresses to the next stage if it satisfies the defined success criteria, ensuring that only thoroughly validated changes reach commuters.

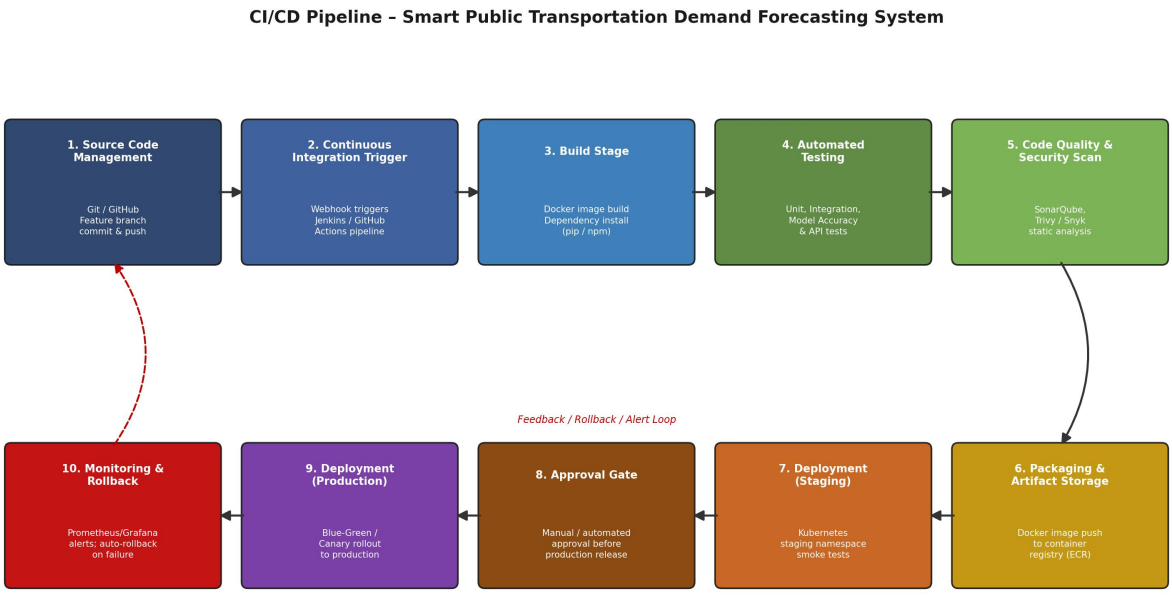

Figure 1: End-to-End CI/CD Pipeline – Smart Public Transportation Demand Forecasting System

3.1 Stage-by-Stage Workflow Explanation

Stage 1 – Source Code Management

Developers and data scientists work on isolated feature branches (e.g., feature/demand-model-v2) in a Git repository hosted on GitHub. Branch protection rules require at least one peer review and a passing status check before merging into the develop branch.

Stage 2 – Continuous Integration Trigger

A push or pull-request merge automatically triggers the CI pipeline via a GitHub Actions workflow (or Jenkins webhook), eliminating manual build initiation and ensuring every change is validated immediately.

Stage 3 – Build Stage

The pipeline builds Docker images for each microservice (data-ingestion service, forecasting-model API, schedule-optimizer, dashboard) and installs dependencies via pip/npm inside isolated, reproducible containers.

Stage 4 – Automated Testing

Unit tests validate individual functions and the ML model's data-preprocessing logic; integration tests confirm that microservices communicate correctly; model-validation tests check the forecasting accuracy (RMSE/MAE) against a held-out historical dataset before the build is allowed to proceed.

Stage 5 – Code Quality & Security Analysis

SonarQube performs static analysis for code smells and maintainability, while Trivy and Snyk scan container images and dependencies for known vulnerabilities. A build failing the quality gate (e.g., Critical CVE or coverage below threshold) is automatically blocked from proceeding.

Stage 6 – Packaging & Artifact Storage

Validated Docker images are tagged with the Git commit hash and semantic version, then pushed to a private container registry (Amazon ECR), creating an immutable, versioned artifact history that supports instant rollback.

Stage 7 – Deployment to Staging

The packaged artifacts are automatically deployed to a Kubernetes staging namespace that mirrors production configuration, where smoke tests, load tests and UAT are performed with anonymized data.

Stage 8 – Approval Gate

A manual approval checkpoint (DevOps lead / project supervisor sign-off) is required before promotion to production, ensuring human oversight for a system that directly affects live public transit operations.

Stage 9 – Deployment to Production

Approved releases are deployed using a Blue-Green or Canary strategy on the production Kubernetes cluster, allowing traffic to shift gradually while health checks are continuously verified.

Stage 10 – Monitoring & Rollback

Prometheus and Grafana continuously monitor API latency, error rates, and model-prediction drift. If any metric breaches its threshold, an automated rollback restores the last stable release and alerts the DevOps team via Slack/email — closing the feedback loop shown by the dashed line in Figure 2.

4.2 Failure Handling at Each Stage

A key design goal of this pipeline is that a failure at any stage should stop the release immediately rather than allow a defective change to progress silently. The table below summarizes the failure response configured for each stage.

Stage	Failure Condition	Automated Response
Build	Docker build error / missing dependency	Pipeline halts; commit author notified via Slack/email with build log link.
Automated Testing	Any unit/integration/model-validation test fails	Merge blocked; PR marked as failing; developer must fix and re-push.
Quality & Security Scan	Critical/High CVE or quality gate breach	Build marked failed; artifact is not promoted to the registry.
Staging Deployment	Smoke test or load test failure	Staging release automatically rolled back; QA and DevOps alerted.
Production Deployment	Health check failure post-rollout	Traffic is not shifted (Blue-Green) or is halted (Canary); auto-rollback triggered.
Post-Deployment Monitoring	Error-rate/latency/drift threshold breach	Kubernetes/Helm rollback to last stable release; incident ticket auto-created.

5. Tools & Technology Selection

The table below lists the tools selected for each stage of the pipeline along with the justification for their selection, based on suitability for an AI/ML-driven, high-availability public transportation platform. The overall toolchain was chosen to favour open-source, widely adopted, and cloud-native technologies, minimizing licensing cost while maximizing community support and integration maturity with Kubernetes-based deployments.

Pipeline Stage	Recommended Tool(s)	Justification
Source Code Management	Git + GitHub (with branch protection)	Industry-standard distributed VCS; supports pull-request review, branch policies, and webhook triggers needed for AI/ML model versioning.
CI Orchestration	Jenkins / GitHub Actions	Free, highly extensible, large plugin ecosystem; GitHub Actions integrates natively with GitHub-hosted repos and requires no separate server maintenance.
Containerization & Build	Docker, Docker Compose	Ensures the AI forecasting service, APIs, and dependencies (TensorFlow/Scikit-learn, GPS/IoT connectors) run identically across dev, staging and production.
Dependency & Package Mgmt	pip / requirements.txt, npm (for dashboard UI)	Standard, reproducible dependency resolution for Python ML services and the Node/React-based operator dashboard.
Automated Testing	PyTest, JUnit, Postman/Newman	PyTest covers ML model unit tests and data-pipeline logic; Postman/Newman automates REST API contract testing for the forecasting endpoints.
Code Quality & Static Analysis	SonarQube, Pylint	Detects code smells, duplication and maintainability issues in Python/JS codebase before merge, enforcing quality gates.
Security Scanning	Trivy, Snyk, OWASP Dependency-Check	Scans Docker images and open-source dependencies for known CVEs; critical since the system processes sensitive commuter/GPS location data.
Artifact Repository	Amazon ECR / Docker Hub (private registry)	Central, versioned storage for built container images, enabling rollback to any previous tagged release.
Configuration Management	Kubernetes ConfigMaps & Secrets, HashiCorp Vault	Externalizes environment-specific configuration (API keys, DB credentials, model version) securely across environments.
Deployment / Orchestration	Kubernetes (EKS/GKE) with Helm charts	Enables declarative, repeatable deployments, auto-scaling for peak-hour demand spikes, and native rolling/blue-green updates.
Monitoring & Alerting	Prometheus + Grafana, ELK Stack	Real-time visibility into API latency, model prediction drift, container health and infrastructure metrics; ELK centralizes logs for

Pipeline Stage	Recommended Tool(s)	Justification
		RCA.
Notification	Slack / MS Teams Webhooks, Email (SMTP)	Immediate pipeline status alerts (build failure, deployment success/failure) to the DevOps and data-science teams.

Together, these tools form a cohesive, cloud-native toolchain: GitHub/GitHub Actions handles source and orchestration, Docker/Kubernetes standardizes runtime environments across all stages, and Prometheus/Grafana closes the loop with observability — ensuring that every change from commit to production is automated, versioned, and monitored.

6. Automated Testing Strategy

A multi-layered automated testing strategy is embedded across the pipeline to ensure that both conventional software correctness and AI-model reliability are validated before every release. This is especially critical for a forecasting-driven system, where an inaccurate model could directly cause commuter overcrowding or wasted fleet capacity.

Testing Type	Scope / Purpose	Tools	Pipeline Stage
Unit Testing	Validate individual functions: data pre-processing, feature engineering, forecasting model output.	PyTest, unittest	Build → Test
Integration Testing	Verify interaction between microservices – data ingestion API, forecasting engine, scheduling module.	PyTest + TestContainers	Test stage
Model Validation Testing	Check prediction accuracy, RMSE/MAE thresholds against historical ridership test set before promotion.	Custom scripts, MLflow	Test stage
API / Contract Testing	Ensure REST endpoints (demand forecast, schedule recommendation) return expected schema and status codes.	Postman/Newman	Test stage
Static Code Analysis	Detect bugs, code smells, security hotspots and enforce coding standards.	SonarQube, Pylint	Quality stage
Security / Dependency Scan	Identify vulnerable libraries and container base-image CVEs.	Snyk, Trivy, OWASP DC	Quality stage
Load / Performance Testing	Simulate peak-hour concurrent requests to validate autoscaling and response time SLAs.	JMeter, k6	Staging
Smoke Testing	Quick sanity check that core endpoints and dashboard load correctly after deployment.	Postman, Selenium	Staging → Prod gate
User Acceptance	Transit operators validate schedule	Manual + Selenium	Staging

Testing Type	Scope / Purpose	Tools	Pipeline Stage
Testing (UAT)	recommendations against real operational scenarios.		

6.1 Testing Pyramid Approach

The strategy follows a testing-pyramid philosophy: a large base of fast, cheap unit tests; a smaller layer of integration and API/contract tests; and a thin top layer of slow, expensive end-to-end/UAT tests. This keeps average pipeline execution time low (target under 10 minutes for CI feedback) while still catching defects at the appropriate layer before they reach staging or production.

6.2 Model-Specific Quality Gate

In addition to conventional software tests, every forecasting-model update must pass an accuracy gate: predictions on a held-out validation dataset must meet or exceed the currently deployed model's RMSE/MAE benchmark. A model that regresses in accuracy is automatically blocked from promotion, preventing a "worse" model from silently replacing a better one in production.

7. Deployment Strategy – Environments

The pipeline promotes code through three progressively production-like environments, with automated and manual gates controlling advancement. This graduated approach minimizes the risk of defective code or an under-performing forecasting model reaching live commuters.

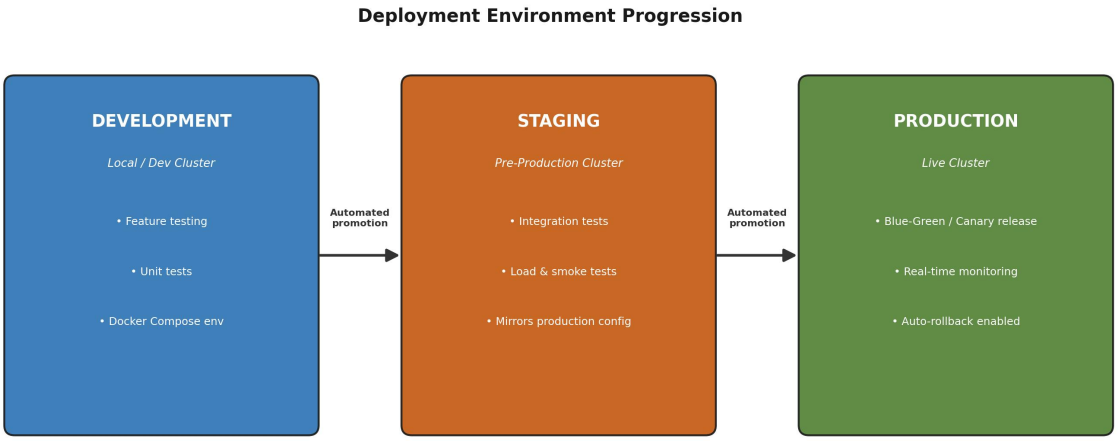

Figure 3: Deployment Environment Progression – Development → Staging → Production

Environment	Purpose	Infrastructure	Deployment Trigger
Development	Feature development and unit testing by individual developers/data scientists.	Local Docker Compose / Dev Kubernetes namespace	Every commit to a feature branch
Staging (Pre-Prod)	Integration, load, security and UAT testing in a production-like environment.	Kubernetes staging namespace, mirrored data schema (anonymized)	Merge to develop / release branch
Production	Live service used by transit operators and the public-facing schedule dashboard.	Kubernetes production cluster with autoscaling, multi-AZ	Manual approval after staging sign-off

7.1 Deployment Techniques Used

- Blue-Green Deployment: two identical production environments allow instant traffic switch-over and equally instant rollback if issues are detected.
- Canary Releases: new forecasting-model versions are exposed to a small percentage of live traffic first, with prediction-drift monitoring before full rollout.
- Rolling Updates: Kubernetes rolling updates ensure the dashboard and API services are updated pod-by-pod with zero downtime.

7.2 Release Cadence

Standard feature releases follow the team's bi-weekly sprint cadence and pass through the full staging → approval → production flow. Critical hotfixes (e.g., a scheduling bug affecting live commuters) use an expedited path with mandatory but time-boxed staging validation, preserving safety without sacrificing response time.

8. Security, Quality Checks, Notifications & Rollback Mechanisms

Because the platform processes commuter location data and directly controls live transit scheduling, security and reliability are treated as first-class pipeline citizens rather than an afterthought. The following practices are embedded throughout the CI/CD workflow.

Security / Reliability Practice	Implementation in Pipeline
Secrets Management	API keys, DB credentials and GPS/IoT tokens stored in HashiCorp Vault / Kubernetes Secrets – never hard-coded or committed to Git.
Dependency & Image Scanning	Trivy and Snyk scan every Docker image and dependency tree at the packaging stage; pipeline fails the build on Critical/High CVEs.
Static Application Security Testing (SAST)	SonarQube security hotspot analysis runs automatically on every pull request before merge.
Role-Based Access Control (RBAC)	Kubernetes RBAC restricts who can trigger production deployments; only DevOps leads can approve the production gate.
Data Privacy Compliance	Passenger location/ridership data anonymized in staging; encryption at rest (AES-256) and in transit (TLS 1.2+).
Blue-Green Deployment	Two identical production environments (Blue/Green); traffic is switched only after health checks pass, enabling instant rollback.
Canary Releases	New model versions are rolled out to 5–10% of traffic first, monitored for prediction drift, before full rollout.
Automated Rollback	Prometheus alerts (error-rate/latency thresholds) trigger Kubernetes/Helm rollback to the last stable release automatically.
Notifications & Audit Trail	Slack/Teams + email alerts on every pipeline stage; all deployments logged with commit hash, approver, and timestamp for auditability.

8.1 Notification Strategy

- Build/Test Failure: instant Slack message to the responsible developer's channel with a direct link to logs.
- Successful Production Deployment: summary posted to the team channel including commit hash, approver, and deployed version.
- Automated Rollback Triggered: high-priority alert (Slack + email + SMS to on-call engineer) with the metric that breached threshold.
- Security Scan Findings: weekly digest of Medium/Low severity issues; Critical/High issues alert immediately and block the build.

9. Sample CI/CD Pipeline Configuration (GitHub Actions)

The following simplified GitHub Actions workflow illustrates how the build, test, quality-scan and deployment stages described above translate into an executable pipeline configuration file (.github/workflows/cicd.yml).

```yaml
name: Smart-Transport-CICD
on:
  push:
    branches: [ develop, main ]
  pull_request:
    branches: [ develop ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker images
        run: docker compose -f build.yml build
      - name: Run unit & integration tests
        run: pytest --cov=app tests/
      - name: Model accuracy validation
        run: python scripts/validate_model.py --threshold rmse=4.5
      - name: SonarQube scan
        run: sonar-scanner
      - name: Container vulnerability scan
        run: trivy image smart-transport:${{ github.sha }}

  push-artifact:
    needs: build-and-test
    runs-on: ubuntu-latest
    steps:
      - name: Push image to ECR
        run: docker push $ECR_REGISTRY/smart-transport:${{ github.sha }}

  deploy-staging:
    needs: push-artifact
    runs-on: ubuntu-latest
    steps:
      - name: Helm upgrade staging
        run: helm upgrade smart-transport ./chart -f values-staging.yaml
      - name: Run smoke tests
        run: newman run staging-smoke-tests.postman_collection.json

  deploy-production:
    needs: deploy-staging
    environment: production   # requires manual approval
    runs-on: ubuntu-latest
    steps:
      - name: Canary rollout via Helm
        run: helm upgrade smart-transport ./chart -f values-prod.yaml --set canary.weight=10
```

This configuration demonstrates the direct mapping between the conceptual pipeline stages of Section 4 and a real, executable automation script — build and test run on every push, artifacts are only pushed after tests pass, staging deployment runs smoke tests automatically, and production deployment is gated

behind an explicit manual approval environment, consistent with the approval-gate design described earlier.

10. Cost & Resource Considerations

Adopting this CI/CD pipeline has direct infrastructure and operational cost implications, which were considered during tool selection to balance robustness with affordability for a public-sector transit deployment.

Resource Category	Estimated Need	Cost-Optimization Approach
CI/CD Compute (build/test runners)	GitHub-hosted runners or self-hosted small EC2 instance	Use GitHub-hosted free-tier runners for open-source-style workloads; scale self-hosted runners only during peak sprint activity.
Container Registry Storage	Versioned images per microservice (~4–5 services)	Lifecycle policy to auto-expire untagged/old images after 90 days, retaining only release-tagged versions.
Staging Environment	Reduced-capacity Kubernetes namespace mirroring production	Run staging on smaller node pool with autoscaling to zero outside business hours.
Production Environment	Multi-AZ Kubernetes cluster with autoscaling	Autoscaling tuned to real commuter demand curves so extra capacity is provisioned only during peak hours.
Monitoring & Logging	Prometheus/Grafana + ELK retention	Retain detailed logs for 30 days, aggregated metrics for 1 year, reducing storage cost while preserving audit trail.

11. Risk Analysis & Mitigation

The table below identifies key risks associated with automating releases for a live public transportation system, along with the mitigation strategy built into the pipeline design.

Risk	Impact	Mitigation
Faulty forecasting model deployed to production	Inaccurate schedules, commuter overcrowding	Model accuracy gate (Section 6.2), canary rollout with drift monitoring, automated rollback.
Vulnerable dependency introduced	Data breach, service compromise	Mandatory Trivy/Snyk scanning at packaging stage; build blocked on Critical/High CVEs.
Deployment causes downtime during peak hours	Service outage affecting commuters	Blue-Green/Canary deployment strategy; production releases scheduled outside peak windows where possible.
Unauthorized or accidental production deployment	Unreviewed change reaches commuters	Mandatory manual approval gate; Kubernetes RBAC restricting deploy permissions to DevOps leads.

Risk	Impact	Mitigation
Pipeline configuration drift between environments	"Works in staging, fails in production" bugs	Infrastructure-as-Code (Helm charts) with environment-specific values files, version-controlled alongside application code.
Alert fatigue from excessive notifications	Real incidents missed among noise	Severity-tiered notification strategy (Section 8.1) with digest-based reporting for low-severity findings.

12. Rubric Compliance Summary

The table below maps each assessment rubric criterion to the corresponding section(s) of this document, confirming complete coverage of all required deliverables.

Rubric Criterion	Weightage	Covered In
Requirement & Pipeline Analysis	15%	Section 2 – Requirement & CI/CD Pipeline Analysis (functional, non-functional needs, goals, assumptions)
Pipeline Architecture & Workflow	25%	Section 3 – System Architecture; Section 4 – Pipeline Architecture & Workflow (10-stage diagram + explanation + failure handling)
Tools & Technology Selection	20%	Section 5 – Tools & Technology Selection (12-row justified tool table)
Automated Testing & Quality Integration	15%	Section 6 – Automated Testing Strategy (9 testing types, testing pyramid, model quality gate)
Deployment, Security & Rollback Strategy	15%	Section 7 – Deployment Strategy; Section 8 – Security, Quality Checks, Notifications & Rollback
Pipeline Diagram & Documentation	10%	Figures 1–3 (architecture, pipeline flow, environment progression) plus Section 9 sample configuration

13. Conclusion

The designed CI/CD pipeline enables the Intelligent AI-Based Smart Public Transportation Demand Forecasting System to be built, tested, secured, and deployed in a fully automated, auditable, and low-risk manner. By integrating automated unit, integration, model-validation and security testing with a graduated Development → Staging → Production deployment strategy — backed by Blue-Green/Canary releases and automated rollback — the pipeline ensures that commuters consistently receive accurate demand forecasts, optimized schedules, and reliable service, while the engineering team can safely and rapidly iterate on the underlying AI models.

Beyond meeting the immediate assessment requirements, this design reflects real-world DevOps best practice: every stage is automated, every failure has a defined response, every deployment is traceable, and every risk identified in Section 11 has a corresponding, concrete mitigation embedded directly in the pipeline rather than left as a manual process. This makes the pipeline both assessment-ready and practically deployable for an actual transit-agency engagement.